Bài tập tuần 05 -06-07: Spring Boot – Spring Data JDBC – Spring Data JPA – Spring Data Mongodb - Spring Data REST– Thymeleaf

Chương 4: Spring Boot – Spring Data JDBC, Spring Data JPA, Spring Data MongoDB, Spring Data Rest

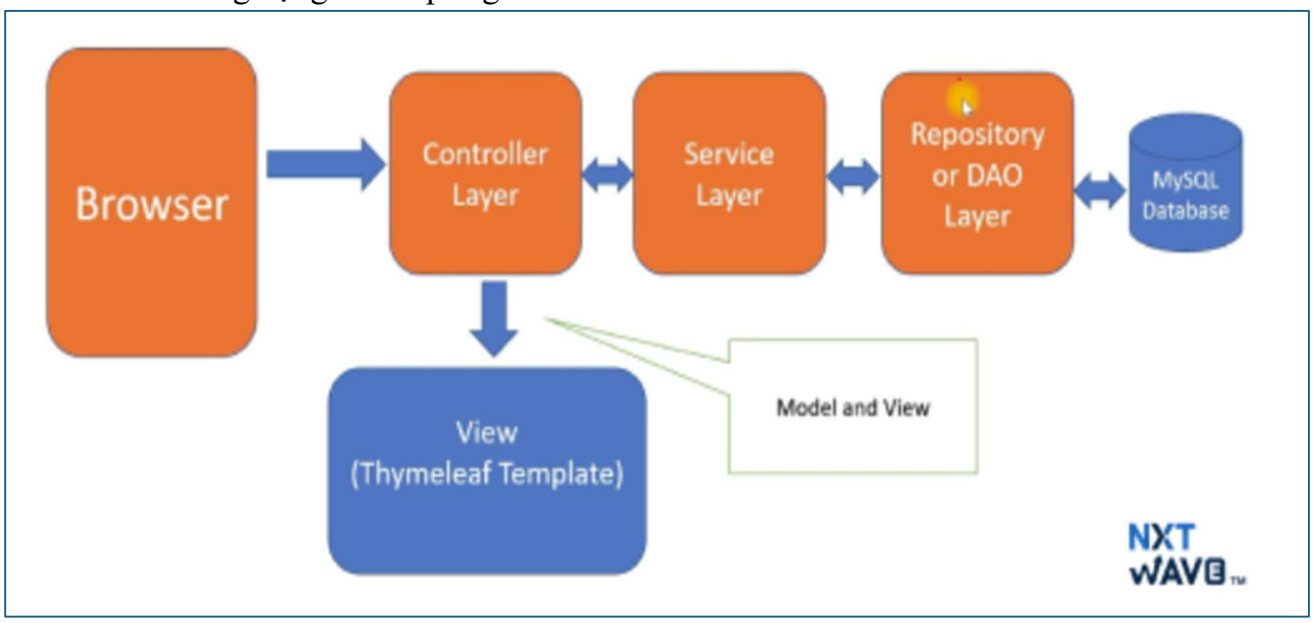
Mục tiêu:

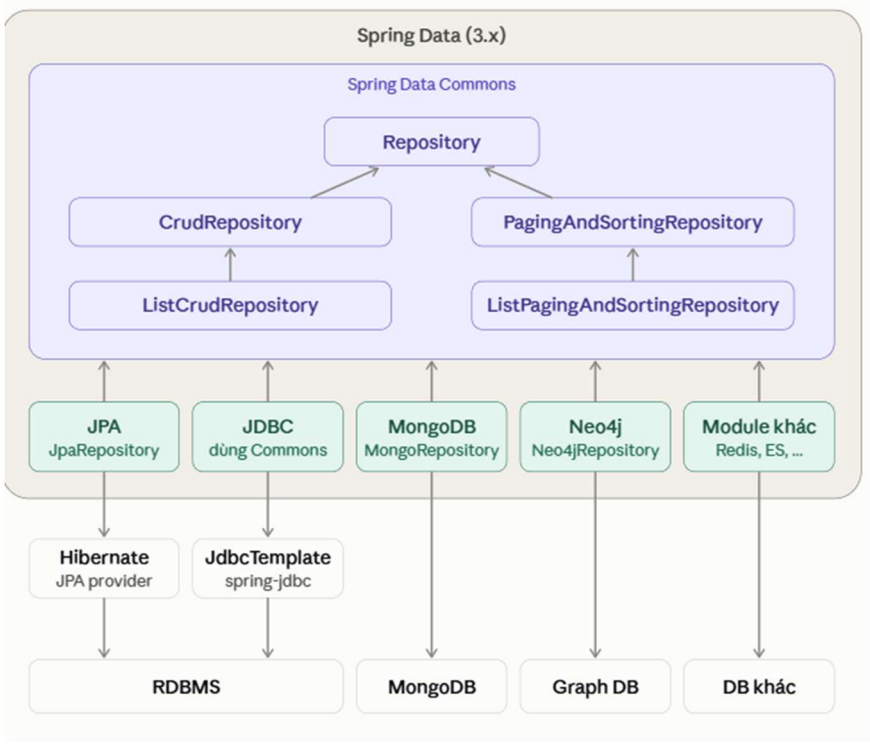
- *Hiểu và áp dụng được cho việc xây dựng ứng dụng Web theo Spring Boot.*
- *Hiểu và áp dụng được kiến trúc MVC xây dựng ứng dụng Web.*
- *Hiểu và áp dụng được 1 trong 3 cách thức Spring Data JDBC, Spring JPA hoặc Spring Mongodb kết nối database thực hiện các thao tác cơ bản CRUD.*
- *Cấu hình kết nối database theo 1 số cách: Config java, application.properties.*
- *Hiểu và sử dụng được các Repository của Spring Boot cung cấp cho việc thực hiện các chức năng cơ bản CRUD*
- *Hiểu về các thức xây dựng RestController và thực hiện RequestMapping, GetMapping*
- *Hiểu và sử dụng Postman để chạy các API cho project Spring Boot*

Yêu cầu:

- *Tất cả các bài tập lưu trong thư mục: T:\MaSV_HoTen_Tuan06*
- *Tạo Project **MaSV_HoTen_Tuan06** trong thư mục trên trong IDE IntelliJ Jakarta EE. **Mỗi bài tập lưu trong từng project riêng biệt***
- *SV phải nén (.rar hoặc .zip) thư mục làm bài và nộp LMS trong buổi đó.*

Kiến trúc của ứng dụng Web Spring Boot

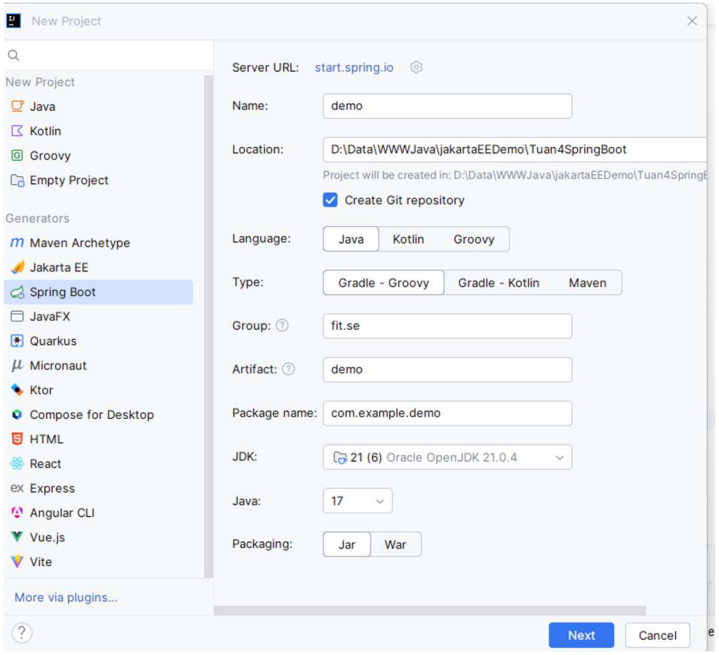




Các bước thực hiện xây dựng ứng dụng Spring Boot trên IDE IntelliJ

Bước 1: Khởi tạo project Spring Boot 3.2+:

- Maven project
- JDK 21
- Tomcat 10



Bước 2: Add các dependencies tương ứng

Bước 3: Thực hiện project theo kiến trúc MVC,

Bài tập về nhà Spring Data:

Yêu cầu 1: Làm bài tập về nhà phần Spring Data, mỗi phần 1 project theo 4 kỹ thuật như sau:

Database SQL *employeedb* gồm 3 table *employee, department, position*

Database NoSQL *employeewww* gồm 3 collection *employee, department, position*

Kỹ thuật	Thuộc về	API chính	Database	Annotation entity
Spring JDBC	Spring Framework	`JdbcTemplate`	SQL (employeedb)	Không cần (dùng RowMapper khi đọc dữ liệu)
Spring Data JDBC	Spring Data	`CrudRepository`	SQL (employeedb)	`@Table, @Id`
Spring Data JPA	Spring Data	`JpaRepository`	SQL (employeedb)	`@Entity, @Id`
Spring Data MongoDB	Spring Data	`MongoRepository, MongoTemplate`	NoSQL (employeewww)	`@Document, @Id`

Yêu cầu 2: Project thực hiện các chức năng:

1. CRUD Employee
2. Các chức năng tìm kiếm Employee theo *id, name, age, salary, position*

Chức năng	Spring JDBC (JdbcTemplate)	Spring Data JDBC	Spring Data JPA	Spring Data MongoDB
Thêm / Sửa	`update("INSERT/UPDATE ...")`	`save(employee)`	`save(employee)`	`save(employee)`
Xóa	`update("DELETE ... WHERE id=?")`	`deleteById(id)`	`deleteById(id)`	`deleteById(id)`
Xem danh sách	`query("SELECT * ...", rowMapper)`	`findAll()`	`findAll()`	`findAll()`
Tìm theo id	`queryForObject(... WHERE id=?)`	`findById(id)`	`findById(id)`	`findById(id)`
Tìm theo name	`WHERE name LIKE ?`	`findByNameContainingIgnoreCase(name)`	`findByNameContainingIgnoreCase(name)`	`findByNameContainingIgnoreCase(name)` hoặc `Criteria.regex`
Tìm theo age	`WHERE age BETWEEN ? AND ?`	`findByAgeBetween(a, b)`	`findByAgeBetween(a, b)`	`findByAgeBetween(a, b)` hoặc `Criteria.gte/lte`
Tìm theo salary	`WHERE salary > ?`	`findBySalaryGreaterThan(s)`	`findBySalaryGreaterThan(s)`	`findBySalaryGreaterThan(s)` hoặc `Criteria.gt`

3. Các chức năng hiển thị thông tin theo mối quan hệ 1 – n:

Chức năng	Quan hệ	Ý tưởng truy vấn (SQL)	Cách triển khai gợi ý
Danh sách Employee theo Position ID	Position 1-n Employee	`WHERE position_id = ?`	JPA/JDBC: findByPositionId(id). JdbcTemplate: query + RowMapper
Danh sách Employee theo Department ID và Position ID	Cả hai quan hệ	`WHERE department_id = ? AND position_id = ?`	findByDepartmentIdAndPositionId(d, p)
Thống kê số Employee của mỗi Department	Department 1-n Employee	`SELECT d.name, COUNT(e.id) FROM departments d LEFT JOIN employees e ON e.department_id = d.id GROUP BY d.id`	@Query + DTO/projection. JdbcTemplate: query + RowMapper cho DTO
Lương trung bình và tổng lương theo Department	Department 1-n Employee	`SELECT department_id, AVG(salary), SUM(salary) FROM employees GROUP BY department_id`	@Query + projection. JdbcTemplate: queryForList hoặc RowMapper
Top N Employee lương cao nhất trong một Department	Department 1-n Employee	`WHERE department_id = ? ORDER BY salary DESC LIMIT N`	findTop5ByDepartmentIdOrderBySalaryDesc(id)
Danh sách Department chưa có Employee nào	Department 1-n Employee	`SELECT d.* FROM departments d LEFT JOIN employees e ON e.department_id = d.id WHERE e.id IS NULL`	@Query (JPA dùng JPQL/native). JdbcTemplate: SQL thuần

Yêu cầu 3: Sử dụng View Thymeleaf thể hiện các chức năng của Website

- Dependency Thymeleaf Spring Boot trong file ***pom.xml***:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

- Cấu hình Thymeleaf trong file ***application.properties***

```
# --- Thymeleaf ---
spring.thymeleaf.cache=false
spring.thymeleaf.mode=HTML
spring.thymeleaf.encoding=UTF-8
spring.thymeleaf.suffix=.html
spring.thymeleaf.prefix=classpath:/templates/
```

- Minh họa Controller các phương thức theo các Request tương ứng thực hiện của project:

EmployeeController.java

```java
package com.example.employeemanagement.controller;

import com.example.employeemanagement.dto.EmployeeForm;
import com.example.employeemanagement.dto.EmployeeSearchCriteria;
import com.example.employeemanagement.entity.Employee;
import com.example.employeemanagement.service.EmployeeService;
import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
@RequestMapping("/employees")
public class EmployeeController {

    private final EmployeeService employeeService;

    public EmployeeController(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }

    // ----- Read: danh sách -----
    @GetMapping
    public String list(Model model) {
        model.addAttribute("employees", employeeService.findAll());
        return "employees/list";
    }

    // ----- Tìm kiếm theo id, name, age, salary, position -----
    @GetMapping("/search")
      public String search(@ModelAttribute("criteria") EmployeeSearchCriteria criteria,
Model model) {
        if (criteria.hasAny()) {
            model.addAttribute("employees", employeeService.search(criteria));
        }
        return "employees/search";
    }

    // ----- Create -----
    @GetMapping("/new")
    public String newForm(Model model) {
        model.addAttribute("employeeForm", new EmployeeForm());
        return "employees/form";
    }

    @PostMapping
    public String create(@Valid @ModelAttribute("employeeForm") EmployeeForm form,
                         BindingResult result, RedirectAttributes redirect) {
        if (result.hasErrors()) {
            return "employees/form";
        }
        Employee saved = employeeService.save(null, form);
         redirect.addFlashAttribute("message", "Đã thêm nhân viên " + saved.getName() +
".");
        return "redirect:/employees";
    }

```

EmployeeController.java

```
57
58      // ----- Read: chi tiết -----
59      @GetMapping("/{id}")
60      public String detail(@PathVariable Integer id, Model model) {
61          model.addAttribute("employee", employeeService.findById(id));
62          return "employees/detail";
63      }
64
65      // ----- Update -----
66      @GetMapping("/{id}/edit")
67      public String editForm(@PathVariable Integer id, Model model) {
68          model.addAttribute("employeeForm", employeeService.getForm(id));
69          return "employees/form";
70      }
71
72      @PostMapping("/{id}")
73      public String update(@PathVariable Integer id,
74                           @Valid @ModelAttribute("employeeForm") EmployeeForm form,
75                           BindingResult result, RedirectAttributes redirect) {
76          form.setId(id);
77          if (result.hasErrors()) {
78              return "employees/form";
79          }
80          Employee saved = employeeService.save(id, form);
81               redirect.addFlashAttribute("message", "Đã lưu thay đổi của nhân viên " +
saved.getName() + ".");
82          return "redirect:/employees";
83      }
84
85      // ----- Delete -----
86      @PostMapping("/{id}/delete")
87      public String delete(@PathVariable Integer id, RedirectAttributes redirect) {
88          employeeService.delete(id);
89          redirect.addFlashAttribute("message", "Đã xóa nhân viên có ID " + id + ".");
90          return "redirect:/employees";
91      }
92  }
93
```